PrestaMatch

Informe final — Trabajo Práctico de Inteligencia Artificial 2026

UCSE DAR — Grupo 11: Arias, Gretter y Molina

1. Problema y objetivo

PrestaMatch busca priorizar solicitudes de préstamos P2P para maximizar la ganancia neta esperada, sujeto a un presupuesto total de USD 2.000.000. Para ello, se considera la probabilidad estimada de pago, el monto solicitado y la tasa de interés de cada solicitud.

2. Datos y preparación

Se utilizaron tres archivos: solicitudes_train.csv (32.416 registros y 8 columnas), solicitudes_test.csv (3.061 registros y 7 columnas) y comportamiento_bancario.csv (661.139 registros y 4 columnas).

El historial bancario se agrupó por ID_Usuario para generar 14 variables agregadas relacionadas con saldos, atrasos, cantidad de meses, proporción de meses con atraso y estadísticas de períodos recientes e iniciales. Los desvíos estándar faltantes se reemplazaron por cero.

Luego, se cruzaron las variables bancarias con las solicitudes mediante ID_Usuario. También se añadieron ratios entre el monto solicitado y el ingreso mensual, entre el saldo promedio y el monto solicitado, el mes de solicitud y variables indicadoras del motivo del préstamo. En total, se utilizaron 27 variables predictoras.

3. Modelos evaluados y selección

Se dividió el conjunto de entrenamiento, reservando el 25 % para validación con estratificación por Estado_Final y una semilla aleatoria fija de 42. Se compararon dos modelos: Regresión Logística, con estandarización, y XGBoost.

Ambos modelos se evaluaron utilizando la misma estrategia de asignación del presupuesto y la ganancia neta obtenida en validación.

Modelo	Ganancia neta en validación
Regresión Logística	USD 591.389,05
XGBoost	USD 489.920,87

Se seleccionó **Regresión Logística**, ya que obtuvo una ganancia neta superior por USD 101.468,18 respecto de XGBoost. La elección se basó en el resultado económico de validación y no únicamente en una métrica de clasificación.

4. Estrategia de asignación del presupuesto

Para cada solicitud se calcula el retorno esperado por dólar mediante la fórmula:

ROI esperado = p × (1 + t) − 1

Donde p representa la probabilidad estimada de pago y t la tasa de interés.

Las solicitudes se ordenan de mayor a menor ROI esperado. Se descartan las que tienen un ROI no positivo y se aprueban aquellas que entran en el presupuesto disponible. Este procedimiento es una estrategia *greedy*: permite realizar una asignación práctica, pero no garantiza encontrar la combinación global óptima del problema de mochila.

5. Resultados finales

El modelo seleccionado se reentrenó con todo el conjunto de entrenamiento y se utilizó para predecir las solicitudes del conjunto de prueba.

- Solicitudes evaluadas: 3.061.
- Solicitudes aprobadas: 639 (20,88 %).
- Presupuesto utilizado: USD 2.000.000 de USD 2.000.000.
- Archivo generado: submission.csv.

Se verificó que submission.csv contiene las columnas ID_Solicitud y Decision_Aprobacion, tiene 3.061 filas y utiliza decisiones binarias (0 y 1).

6. Enfoques alternativos y limitaciones

La alternativa efectivamente evaluada fue XGBoost. Aunque es un modelo más flexible, obtuvo una ganancia neta menor que Regresión Logística en la validación, por lo que no fue seleccionado para la predicción final. No se documentan comparaciones experimentales con otros algoritmos en esta versión del pipeline, por lo que no se presentan como métodos probados.

La ganancia de validación es una estimación sobre una partición de los datos y no garantiza el mismo resultado en futuras solicitudes. Además, la estrategia greedy basada en ROI prioriza el retorno porcentual, pero puede producir una selección diferente de la que se obtendría ordenando por ganancia absoluta. Tampoco garantiza una solución exacta al problema de mochila.

7. Reproducibilidad y entrega

Se fijó random_state=42 en la división de datos y en los modelos que lo admiten. El pipeline carga los datos, genera las variables, compara los modelos, selecciona el de mayor ganancia de validación, lo reentrena con todos los datos de entrenamiento y genera submission.csv.

El repositorio debe incluir el código fuente, requirements.txt, este informe y los archivos de datos necesarios para ejecutar el proceso.